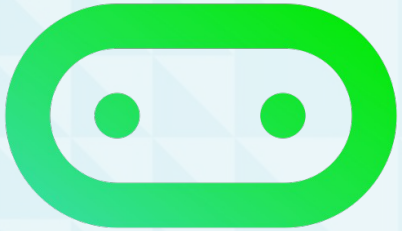


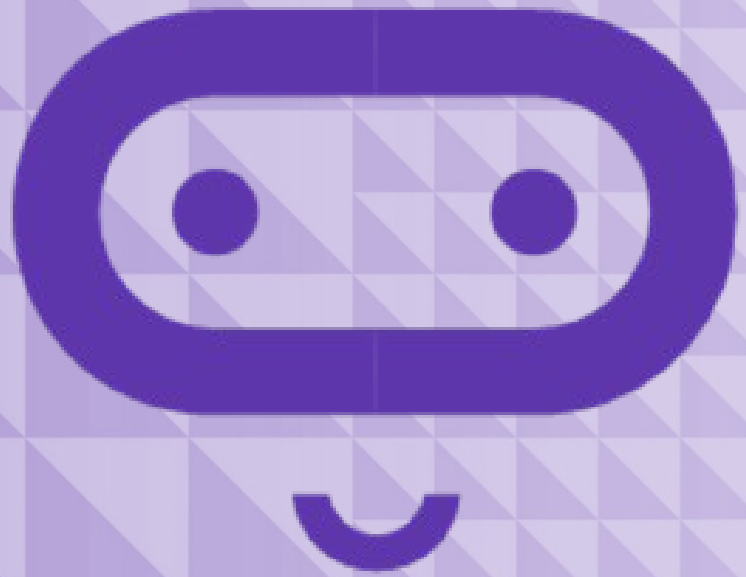
micro:bit

Introduction to Computer Science

Unit 11: Arrays

Microsoft

Lesson A
Understanding arrays



What we'll learn in Lesson A

- What arrays are
- How to sort arrays

Arrays

An **array** is a list, or a collection, of similar things—like a rock or coin collection.

What are some examples of arrays in everyday life?

In programming, you can think of arrays as a list of individual variables with the same data type.

Arrays can store **numbers**, **strings (words)**, **sprites**, and even musical notes.

Arra

Every
by its **i**
corres

The **fi**

The **le** …th
of our

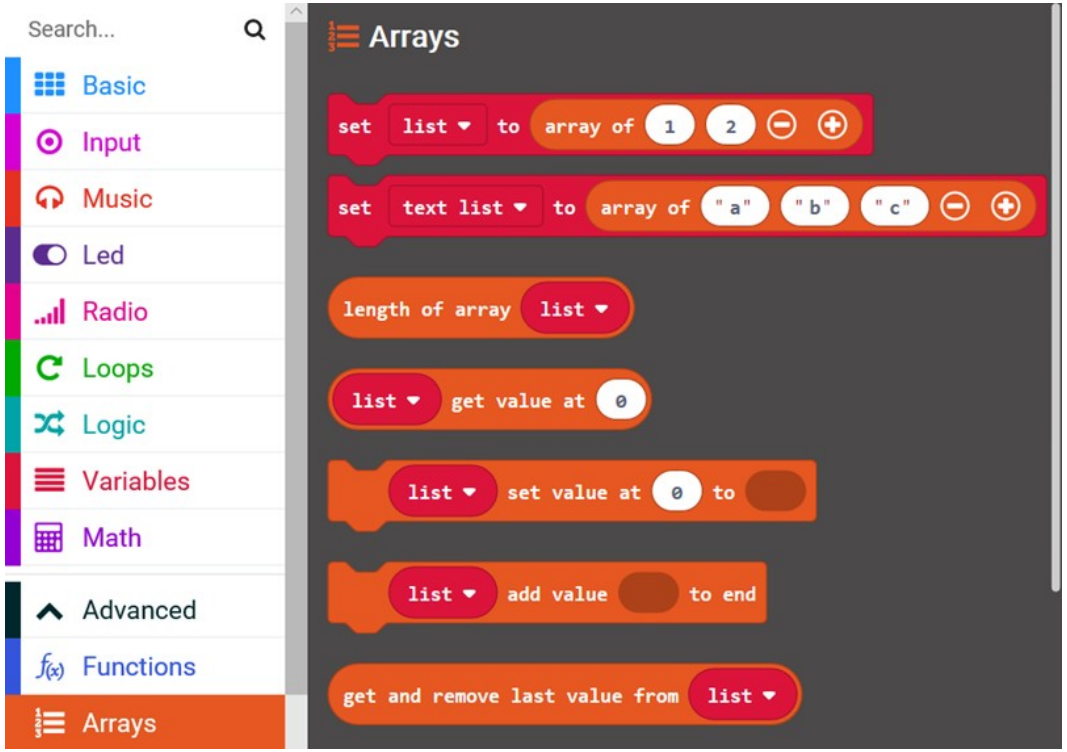
The **i**
(because the array numbering starts at zero.)

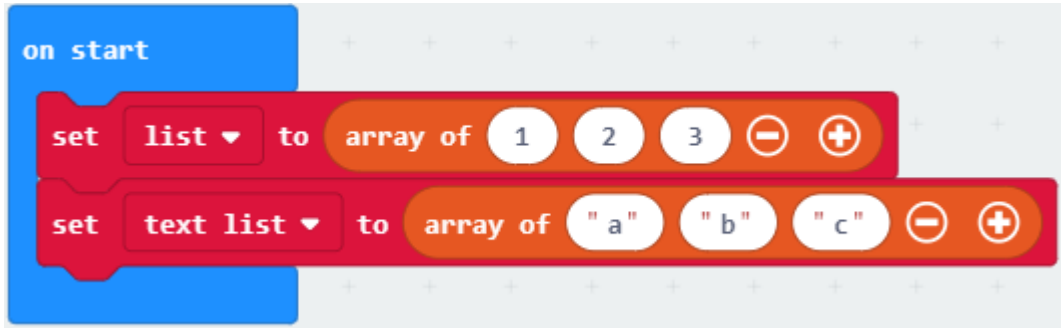
What is the index of the last element in our rock collection here?

Arrays in MakeCode

In MakeCode, you can find the Array blocks in the **Advanced Toolbox** area.

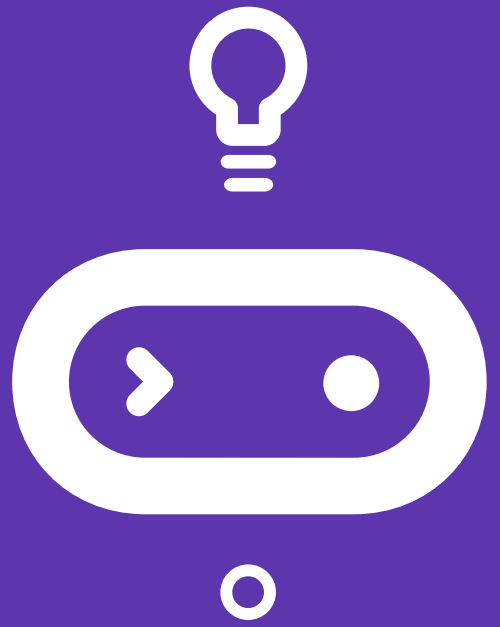
Use **the set blocks** to create arrays of numbers or strings.





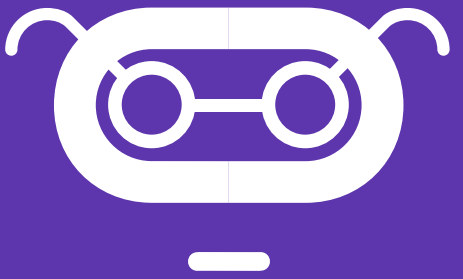
Let’s discuss

- Who collects something? What do you collect?
- How big is the collection?
- How is it organized?
- Are the items sorted in any way?
- How would you find a particular item in the collection?



Let’s review

- **Length**: the total number of items in the collection
- **Sort**: Items in the collection are ordered by a particular attribute (e.g., date, price, name)
- **Index**: A unique address or location in the collection
- **Type**: The type of item being stored in the collection (number, string, etc.)



Different sorts of people

Sortees: Stand at the front of the classroom with your numbers facing out front

Sorter: Place the students in order by directing them to move, one at a time, to the proper place.

Sorting Algorithms

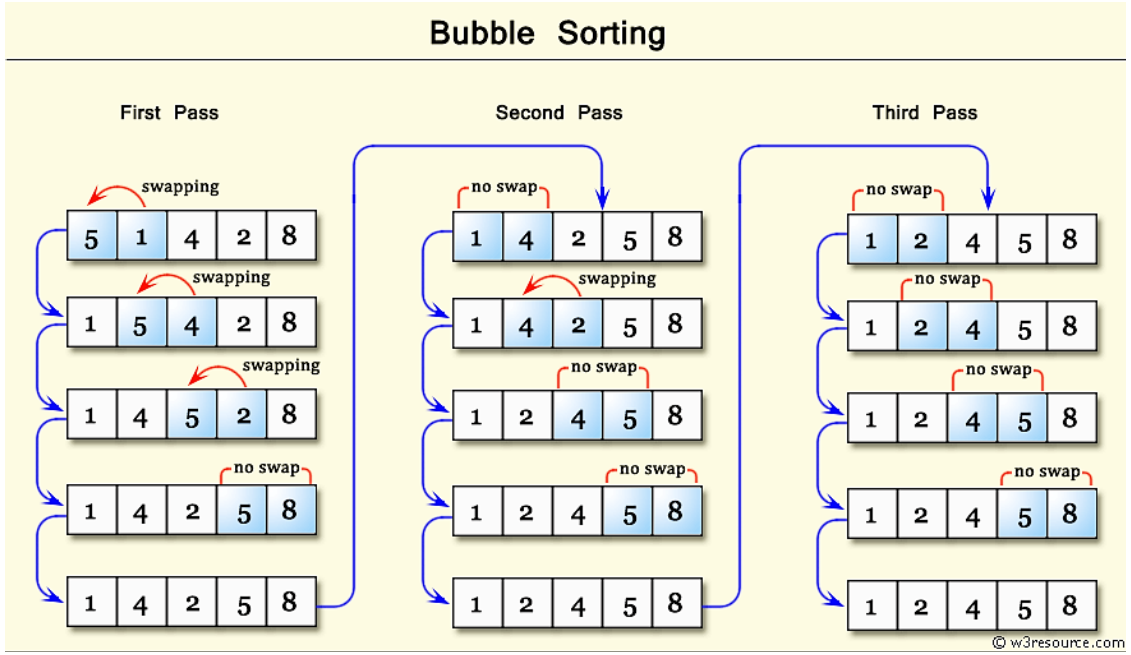
In Computer Science, there are some well-known Sorting Algorithms:

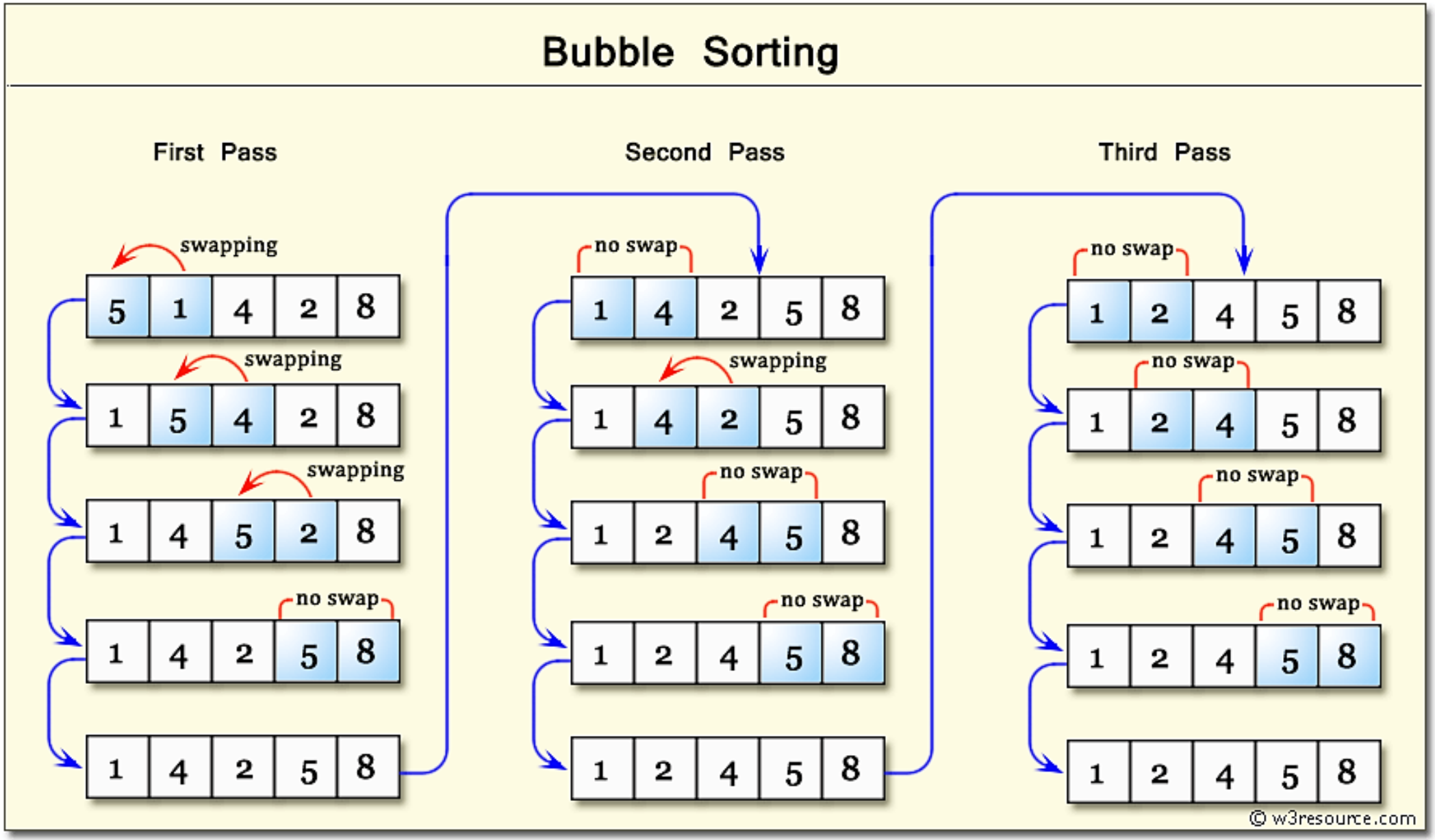
- **Bubble Sort**
- **Selection Sort**
- **Insertion Sort**

Bubble sort

1. Compare the first two students. If the student on the right has a smaller number than the student on the left, they should swap places.
2. Next, compare the second and third students. If the student on the right has a smaller number than the student on the left, they should swap places.
3. When you reach the end, start over at the beginning again.
4. Continue in this way until you make it through the entire row of students without swapping anybody.

Example in MakeCode

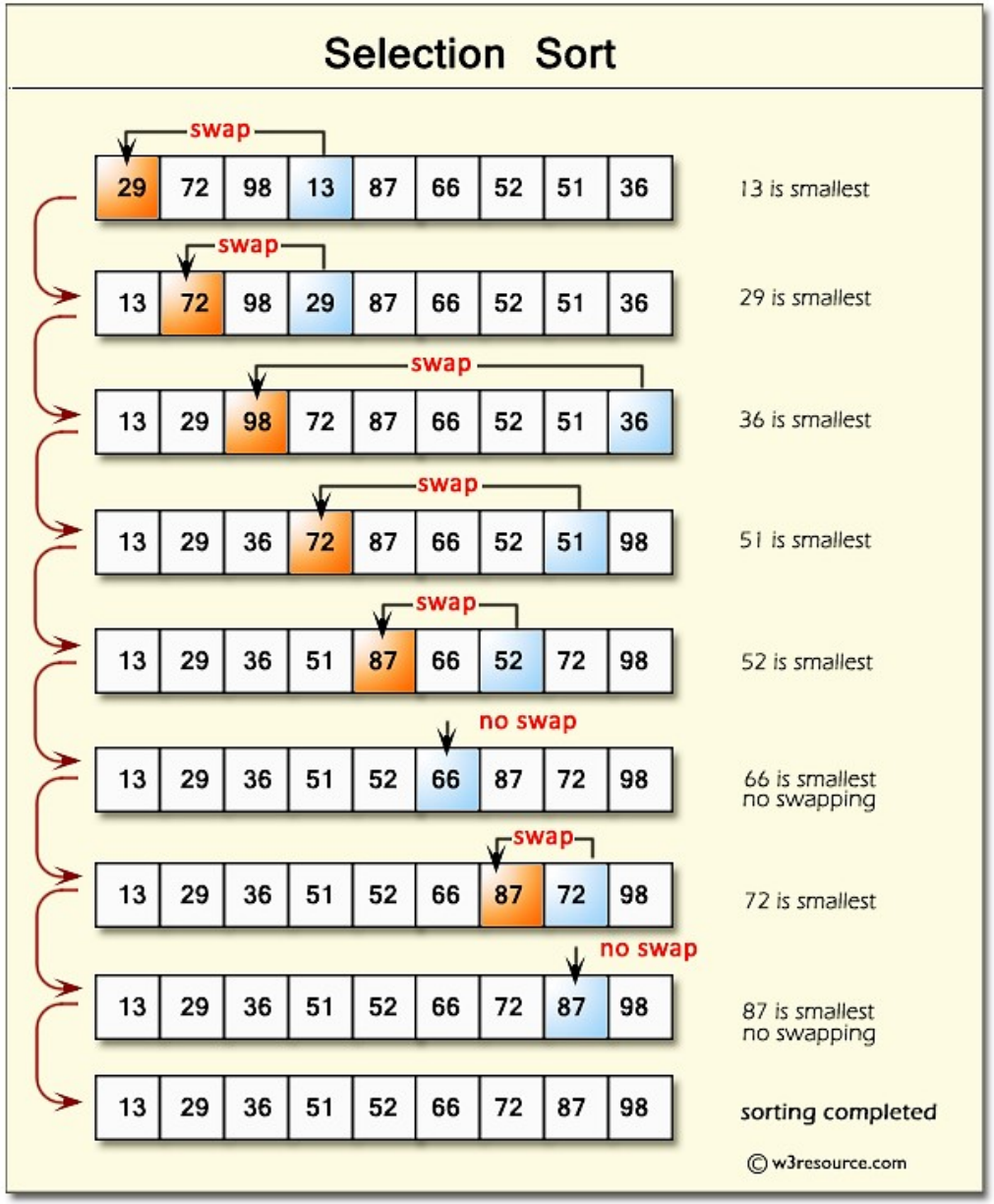



Bubble Sorting
First Pass
Second Pass
Third Pass
swapping
swapping
swapping
no swap
5 1 4 2 8
1 5 4 2 8
1 4 5 2 8
1 4 2 5 8
1 4 2 5 8
no swap
swapping
no swap
no swap
1 4 2 5 8
1 4 2 5 8
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8
no swap
no swap
no swap
no swap
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8
1 2 4 5 8
© w3resource.com

Selection sort

1. Take the first student on the left and consider that person's number the largest number you have found so far.
2. If the next person in line has a number that is smaller than that number, then make that person's number your new smallest number and continue in this way until you reach the end of the line of students.
3. Move the person with the smallest number all the way to the left.
4. Start over from the second person in line.
5. Keep going, finding the smallest number each time, and making that person the rightmost person in the sorted line of students.

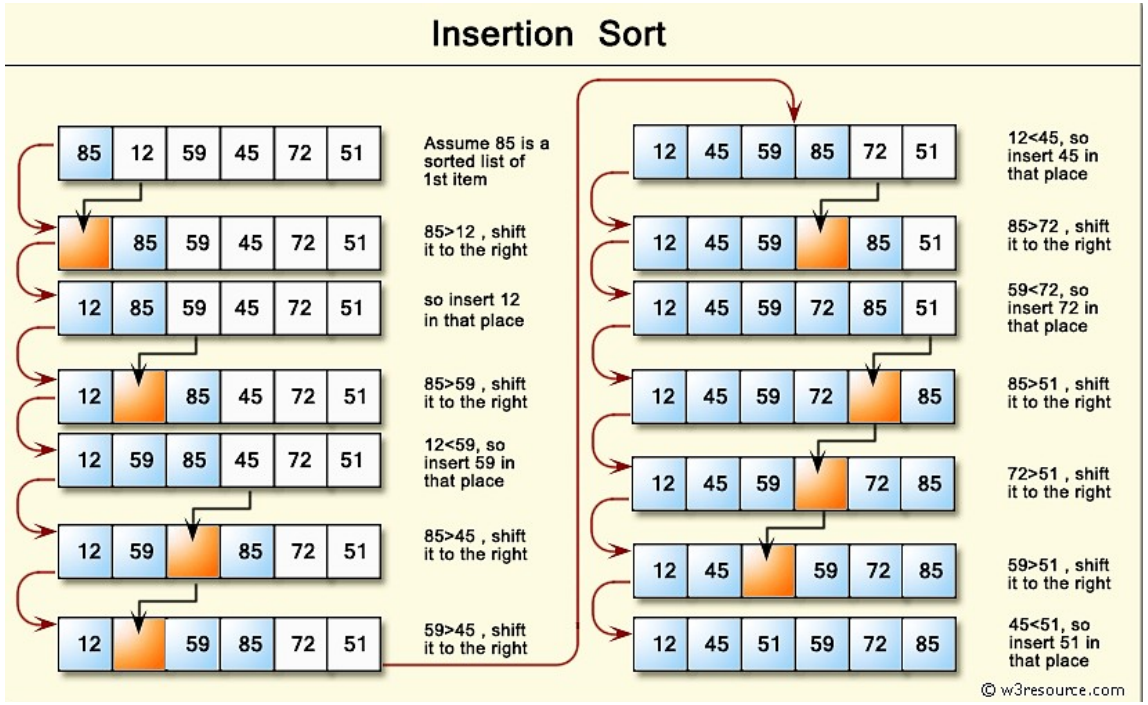
Example in MakeCode

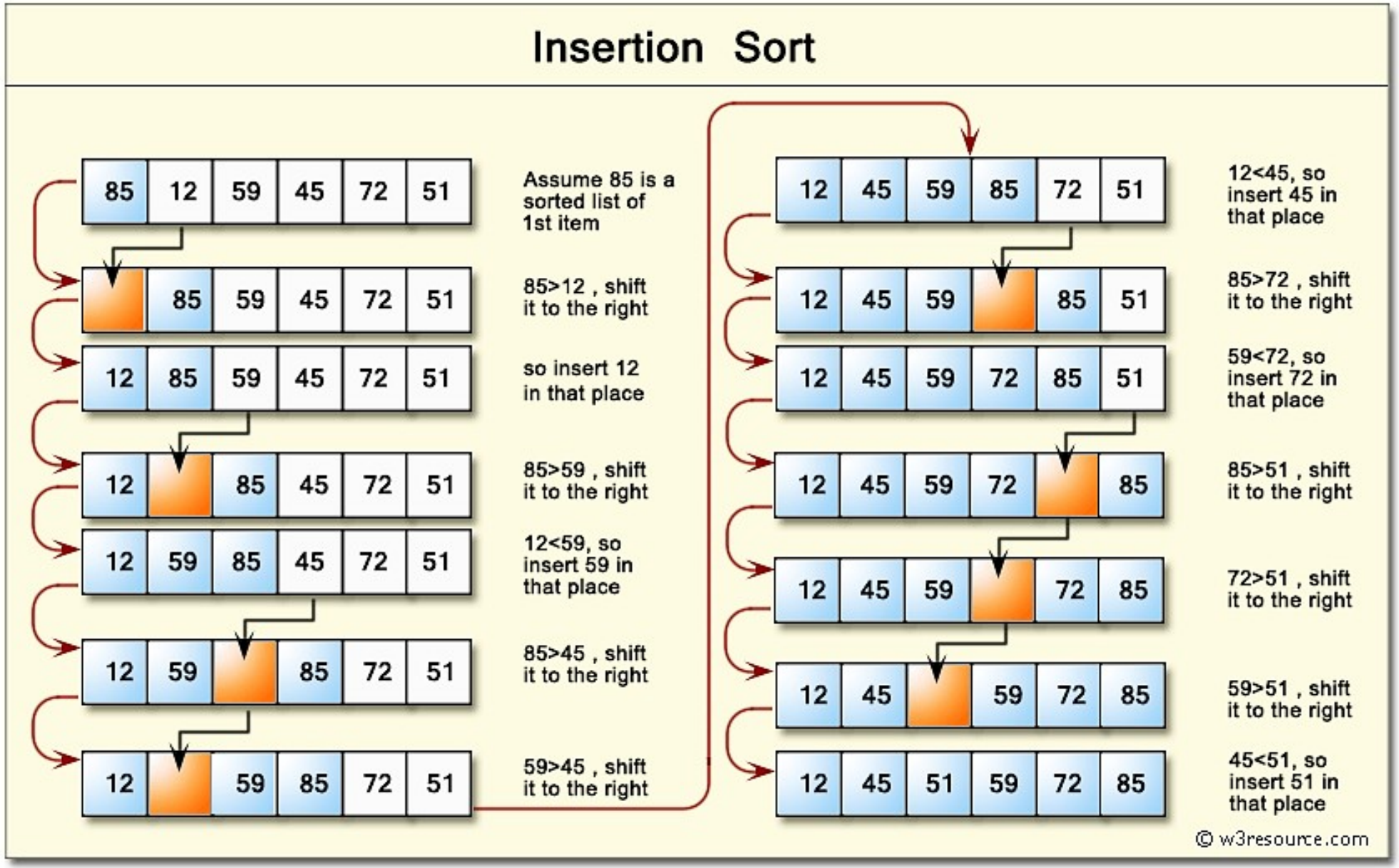


Insertion sort

1. Take the first student on the left and consider that person sorted.
2. Take the next student and compare their number to the first student in the sorted section. If they are greater than the first student, then place them to the right of the student in the sorted section. Otherwise, place them to the left of the student in the sorted section.
3. Continue down the line, considering each student in turn and then moving from left to right along the students in the sorted section until you find the proper place for each student to go, shifting the other students to the right to make room.

Example in MakeCode




Insertion Sort
85 12 59 45 72 51
Assume 85 is a sorted list of 1st item
85 59 45 72 51
85>12 , shift it to the right
12 85 59 45 72 51
so insert 12 in that place
12 85 45 72 51
85>59 , shift it to the right
12 59 85 45 72 51
12<59, so insert 59 in that place
12 59 85 72 51
85>45 , shift it to the right
12 59 85 72 51
59>45 , shift it to the right
12 45 59 85 72 51
12<45, so insert 45 in that place
12 45 59 85 51
85>72 , shift it to the right
12 45 59 72 85 51
59<72, so insert 72 in that place
12 45 59 72 85
85>51 , shift it to the right
12 45 59 72 85
72>51 , shift it to the right
12 45 59 72 85
59>51 , shift it to the right
12 45 51 59 72 85
45<51, so insert 51 in that place
© w3resource.com

What we learned in Lesson A

- Arrays and different methods of sorting arrays

Next time: Lesson B

- Code with arrays
- Assess our learning with a quiz

Lesson B
Coding with arrays

What we'll learn in Lesson B

- Use arrays to store both string and numeric elements
- A special loop block for arrays
- Review how much we've already learned

Headband charades

Create an array of words that will be used as prompts for a Charades game.

This game takes inspiration from the Ellen DeGeneres Heads Up! phone game.

Pseudocode:

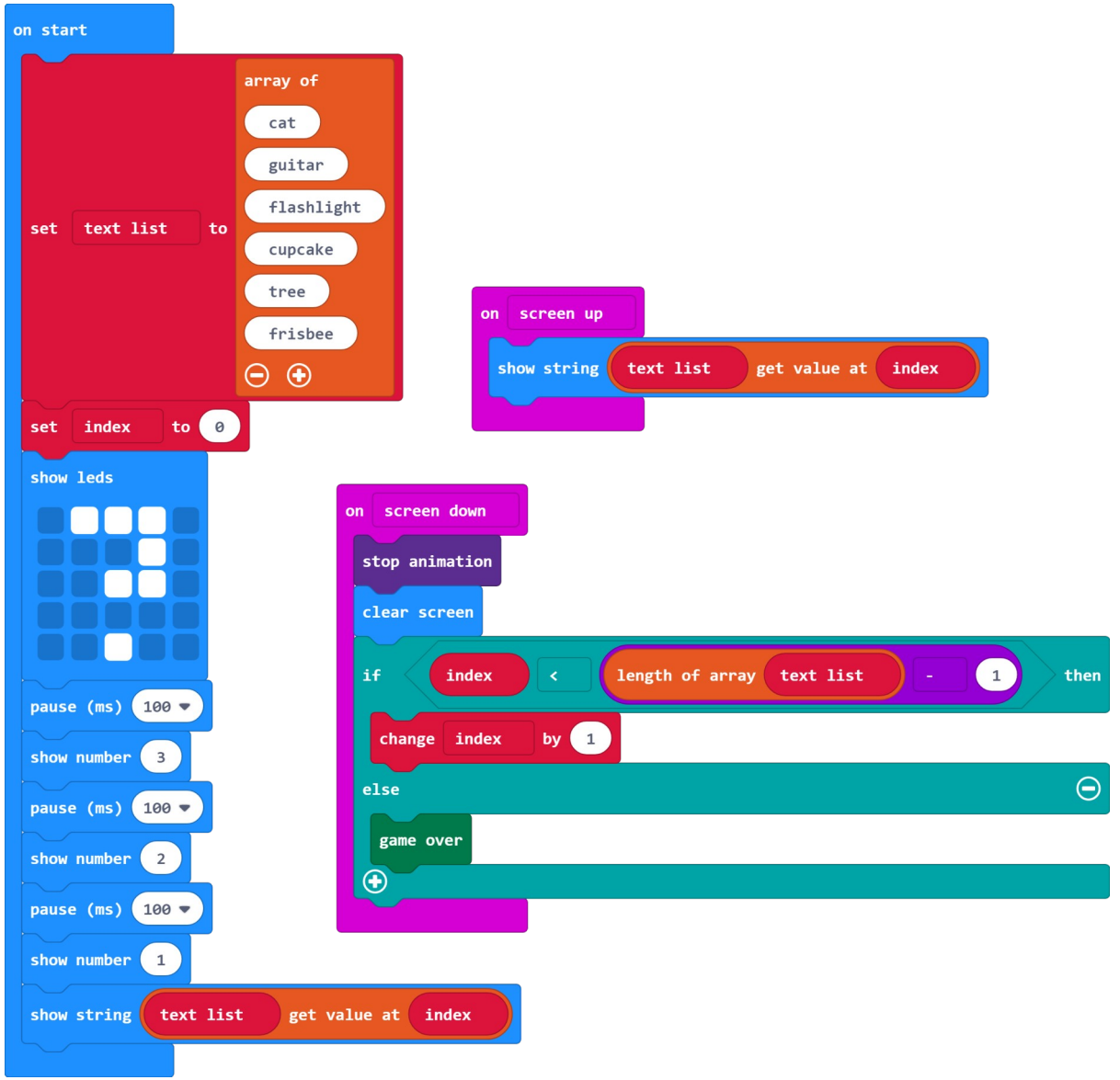
1. Create an array of six random words for the charades prompt
2. When you tilt the micro:bit up, show the word at the current index
3. When you tilt the micro:bit down, increment the array index

Headband charades code

Your final program may look like:

Mods:

- Add a headband to hold the micro:bit on the Players' foreheads (using cardboard, paper, rubber bands, etc.)
- Add a way to keep score
- Keep track of the number of correct guesses and passes
- Add a time limit



Starry, starry night

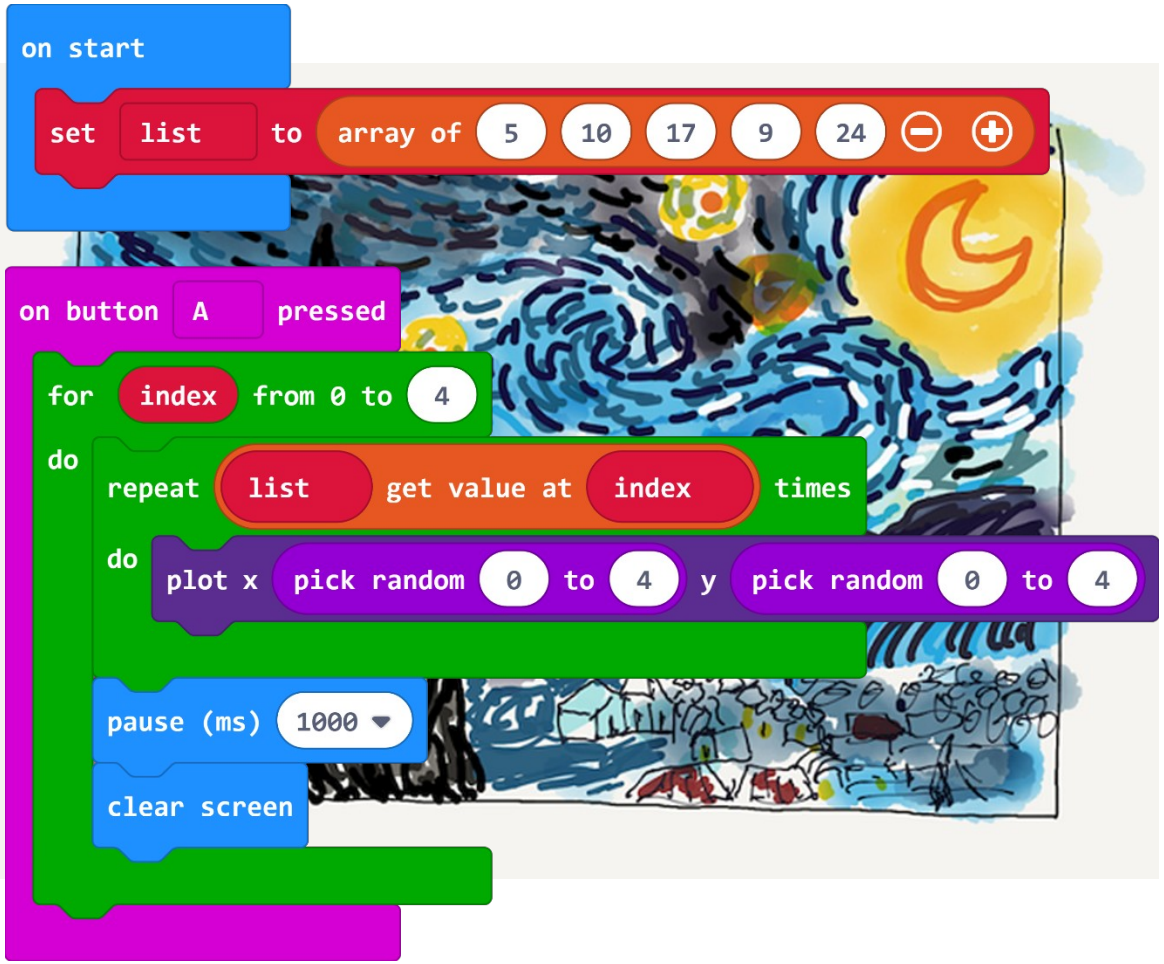
- Create a set of random constellations on the micro:bit screen.
- Use an array of numbers to determine how many ‘stars’ or LEDs to turn on.

Starry, starry night code

Hints:

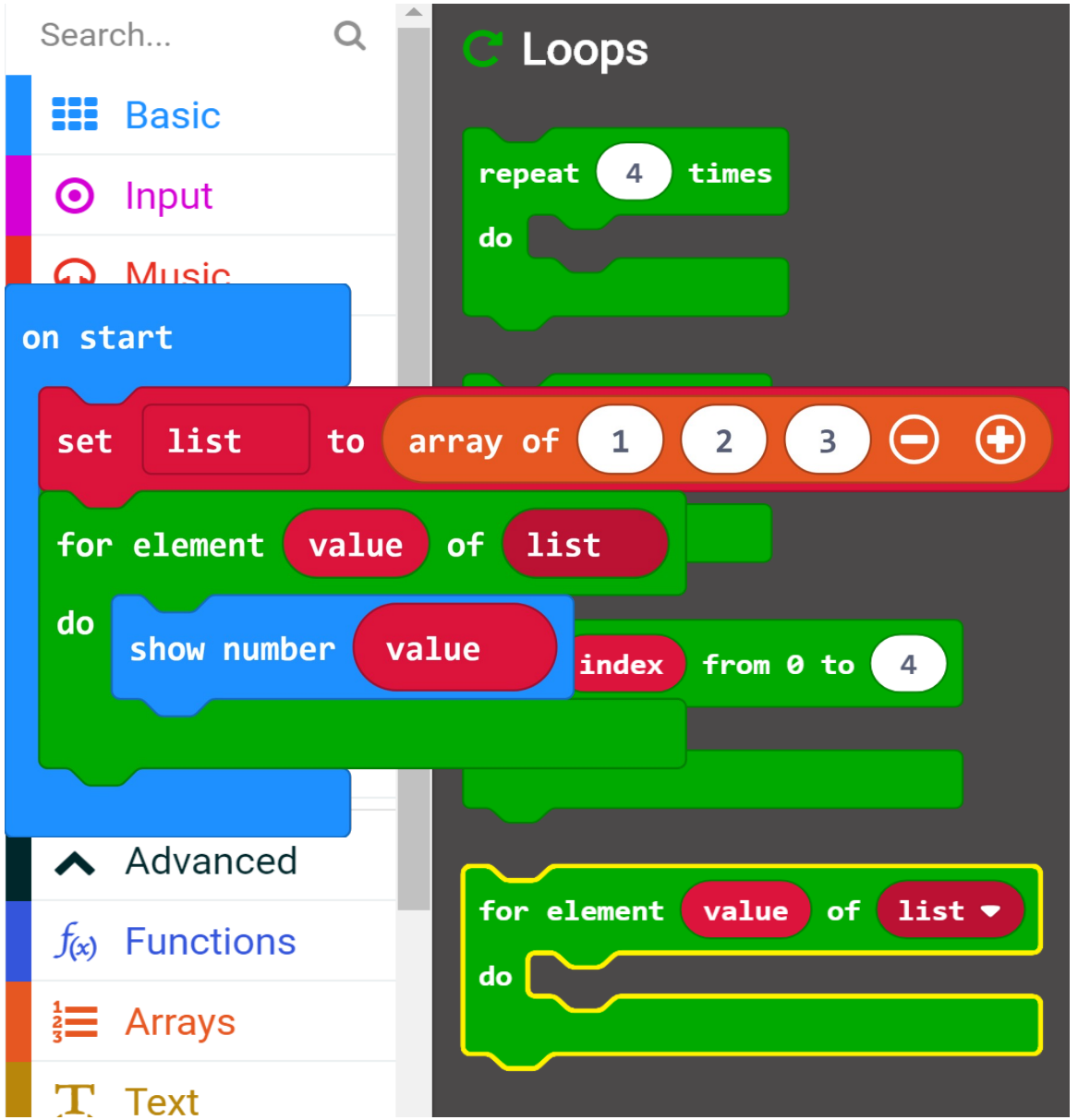
- Use a **For loop** from 0 to the last index value in your array to move through your array of constellations
- Use the **Plot block** with the **Pick Random blocks** to turn on a random ‘star’ or LED on the micro:bit
- Use a Repeat loop to turn on the correct number of stars for each constellation

Your final program may look like:

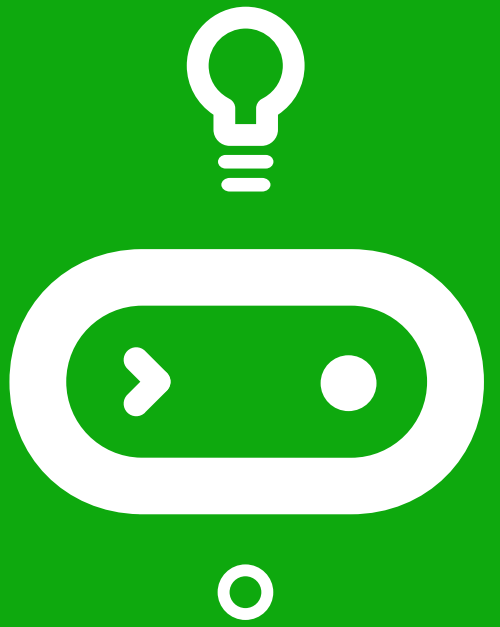


Traversing an array

Instead of using the regular ‘for loop’ block, MakeCode provides a **special ‘for loop’ block for arrays**.



Time for a quiz!



What we learned in Lesson B

- Used arrays to store both string and numeric elements
- How much we've already learned!

Next time: Lesson C

- Code and make a micro:bit musical instrument!

Lesson C

Make a micro:bit musical instrument

What we'll learn in lesson C

- Create your own micro:bit musical instrument using arrays

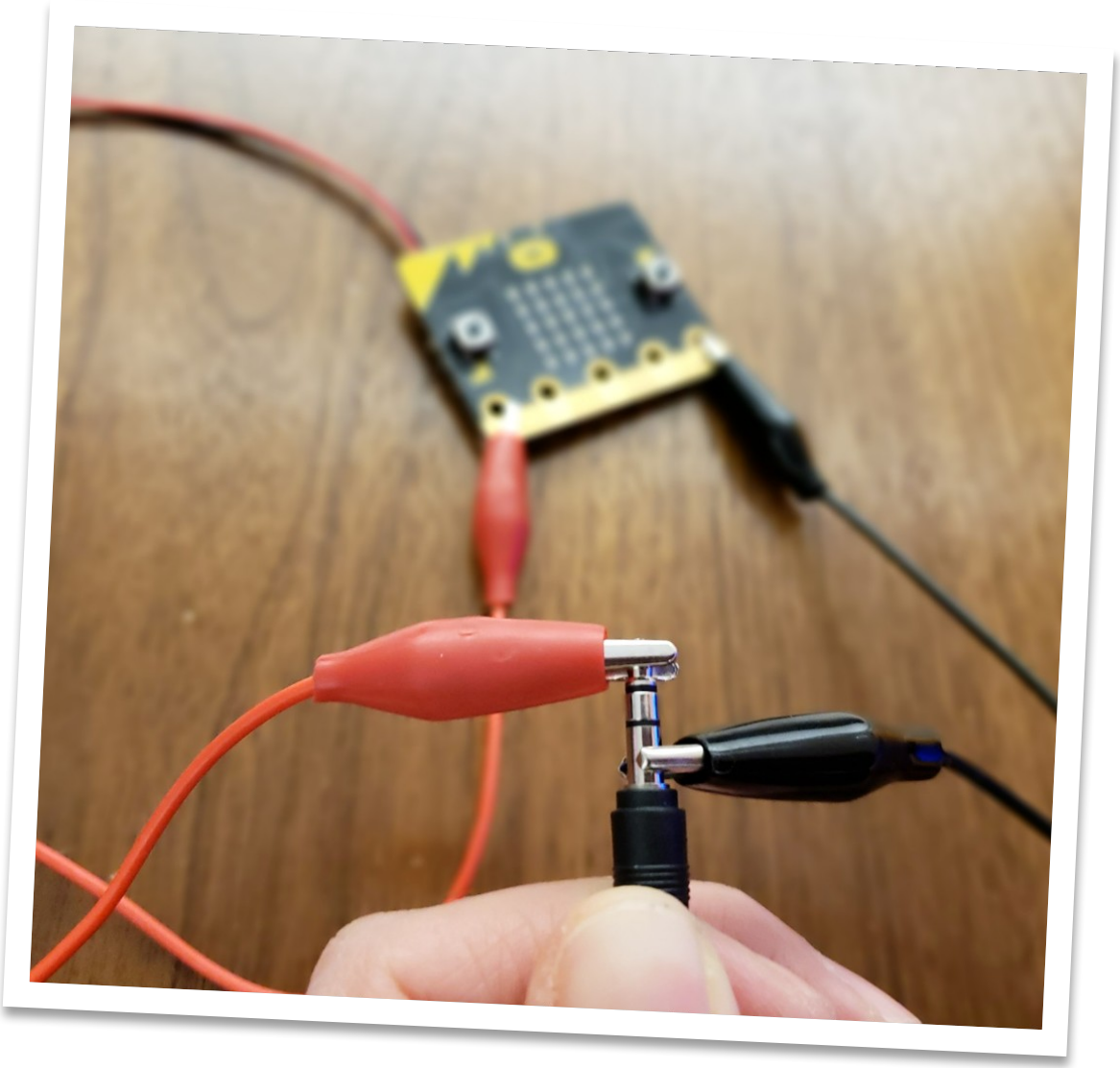
Micro:bit musical instrument

Create a musical instrument that uses arrays to store sequences of notes.

The array of notes can be played when an input occurs, such as one of the buttons being pressed or if one or more of the pins is activated.

Ideally, the micro:bit should be mounted in some kind of housing, perhaps a guitar shape or a music box.

You may need to connect your micro:bit to headphones or a speaker to play sounds.



Example: micro:bit Guitar

Micro:bit guitar

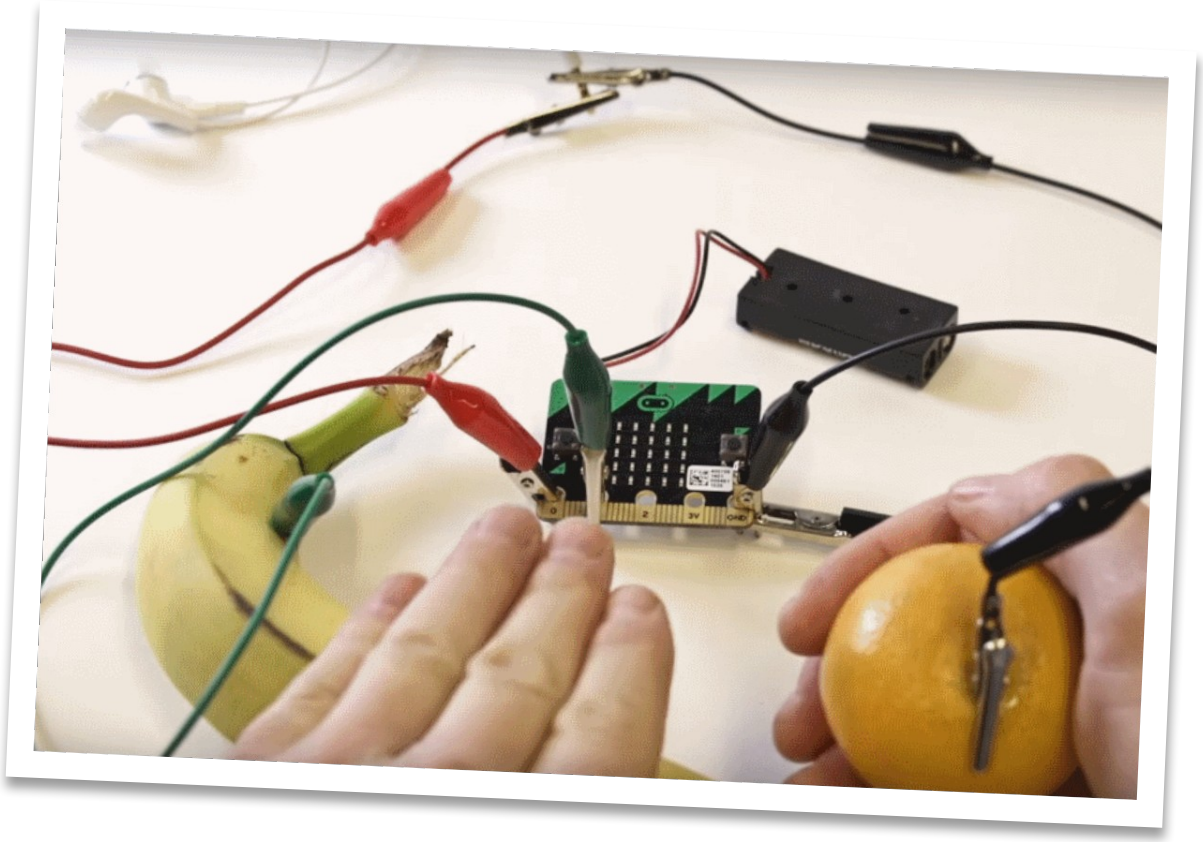
This uses the micro:bit accelerometer to play different tones when the guitar is held and tilted while playing.

- Pressing the A button will save the current tone to an array.
- After ten tones, a repeating melody will be performed.
- Press the B button to clear the array and start over.

Example: Banana keyboard

Banana keyboard

This uses the pins connected to pieces of fruit to activate the music.



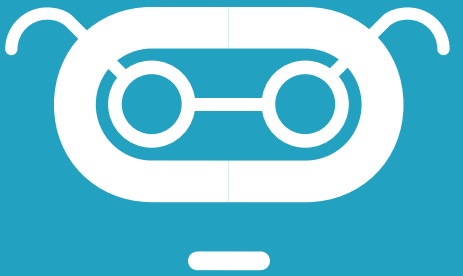
Project expectations

Use a design thinking approach and ensure the project meets these specifications:

- Uses at least one array in a fully integrated and meaningful way, and the array stores and iterates through each element of the array successfully
- The tangible instrument is tightly integrated with the micro:bit and each relies heavily on the other to make the project complete
- The program compiles and runs as intended and includes meaningful comments in code
- Provide the written Reflection Diary entry

Reflection Diary

- Explain how you decided on your musical instrument. What was your inspiration instrument? What brainstorming ideas did you come up with?
- What properties does it share with a real musical instrument? What properties are unique?
- Describe the type of array you used (Numbers, Strings, or Notes) and how it functions in your project.
- What was something that was surprising to you about the process of creating this program?
- Describe a difficult point in the process of designing this program and explain how you resolved it.
- What feedback did your testers give you?
  How did that help you improve your musical instrument?
- Publish your MakeCode program and include the link.



What we learned in Lesson C

- Made a micro:bit musical instrument